

웹 POS 데이터베이스 설계 이해 가이드

지금까지 설계한 식당 POS · QR 주문 · 테이블 운영 · 결제 · 할인 · 재고 · 매입
구조를 실제 개발 요청에 활용할 수 있도록 정리한 문서

핵심 설계 철학

테이블, 손님, 주문, 계산, 결제, 재고를 하나의 큰 표에 섞지 않고 각각의 역할로 분리한다. 대신 고유 ID와 관계를 통해 연결한다. 현재 상태는 빠르게 조회하면서도 과거에 무슨 일이 있었는지 추적할 수 있도록 이력을 남긴다.

대상 DB: MariaDB 10.11 계열 / utf8mb4

설계 범위: 직원, 테이블 세션, 일행 그룹, 메뉴, 다국어, 옵션, 구성품, QR 주문, 주문, 계산, 복합결제, 후불식권, 할인/쿠폰, 재고, 레시피, 매입, 거래처 미지급, 감사로그

목차

1. 전체 구조와 기본 설계 원칙
2. 테이블과 손님 세션
3. 일행 그룹과 테이블 이동
4. 주문과 주문 품목
5. 메뉴 · 카테고리 · 다국어 · 이미지
6. 옵션과 자동 구성품
7. 주방/홀 조리 전송
8. QR 주문 흐름
9. 계산 · 부분계산 · 복합결제
10. 후불식권 · 할인 · 쿠폰
11. 재고 · 레시피 · 재고원장 · 실사
12. 매입 · 거래처 · 외상대금
13. 직원 · 로그인 · 감사로그
14. 실제 손님 한 팀의 전체 흐름
15. 웹 POS 개발 시 요청 문장 예시
16. 반드시 지켜야 할 핵심 원칙

1. 전체 구조와 기본 설계 원칙

이 POS의 중심 흐름은 ‘ 물리 테이블 → 손님 이용 세션 → 주문 → 주문 품목 → 계산 → 결제 ’ 이다. QR 주문, 재고, 매입, 쿠폰 등은 이 중심 흐름에 연결되는 별도 기능이다.

```
dining_tables
↓
table_sessions
↓
orders
↓
order_items
↓
checkout_items
↓
checkouts
↓
payments
```

이렇게 분리하는 이유는 주문과 결제, 테이블과 손님, 메뉴와 재고가 서로 같은 개념이 아니기 때문이다. 각 기능을 독립적으로 만들면 테이블 이동, 추가주문, 부분계산, 복합결제 같은 실제 식당 상황을 훨씬 자연스럽게 처리할 수 있다.

기본 데이터 원칙

테이블명은 복수형 snake_case, 필드명도 snake_case를 사용한다. PK는 단순 id보다 order_id, session_id, payment_id처럼 의미가 드러나게 한다. 시간은 *_at, 사용 여부는 is_* 형태로 통일한다.

과거 주문 · 결제 · 매입 · 재고거래 · 쿠폰사용 기록은 가능한 한 물리적으로 삭제하지 않는다. 취소 상태와 취소시간, 취소자를 남겨서 ‘ 없었던 일 ’ 이 아니라 ‘ 발생했다가 취소된 일 ’ 로 보존한다.

핵심: 이 DB는 단순히 현재 화면을 저장하는 DB가 아니라, 식당에서 발생한 업무의 흐름과 이력을 남기는 DB이다.

2. 테이블과 손님 세션

dining_tables = 실제 식당의 물리 테이블

1번, 2번, 3번 테이블처럼 매장에 실제 존재하는 자리를 등록한다. table_no, table_name, capacity, sort_order, is_active 등이 들어간다.

table_sessions = 한 팀이 그 테이블을 사용한 기록

오늘 점심 1번 테이블 손님과 오늘 저녁 1번 테이블 손님은 같은 테이블을 사용했지만 서로 다른 손님이다. 따라서 주문을 table_id에 직접 붙이지 않고, 손님이 앉을 때 새로운 table_session을 만든다.

```
점심
table_id = 1
session_id = 1001
status = OPEN
opened_at = 12:03
```

```
계산 후
status = CLOSED
```

```
저녁 새 손님
table_id = 1
session_id = 1057
status = OPEN
```

주문은 table_id가 아니라 session_id를 참조한다. 이것이 테이블 이동과 과거 매출 조회를 쉽게 만드는 핵심이다.

3. 일행 그룹과 테이블 이동

테이블 이동

1번 테이블 손님이 5번으로 이동해도 기존 주문의 테이블 번호를 일일이 수정하지 않는다. OPEN table_session의 table_id만 1에서 5로 변경한다. 모든 주문이 같은 session_id를 참조하기 때문에 자동으로 새 자리의 주문처럼 보인다.

party_groups = 여러 테이블이 같은 일행일 때

8명이 1번과 2번에 나누어 앉았다면 각 테이블의 table_session은 별개지만 동일한 group_id를 줄 수 있다. POS 화면에서는 같은 group_id를 가진 OPEN 세션을 같은 색이나 테두리로 표시하면 된다.

```
party_group 50
├─ 1번 table_session → group_id 50
└─ 2번 table_session → group_id 50
```

일행 전체 계산은 같은 group_id의 미결제 주문 품목을 모아 하나의 checkout으로 만들면 된다.

4. 주문과 주문 품목

orders = 한 번의 주문 제출

첫 주문과 추가 주문을 하나의 주문에 계속 덧붙이지 않는다. 손님이 다시 주문할 때마다 새로운 orders 행을 만든다.

```
session 500
├─ order 100 / 12:10
|   ├── 김치찌개 ×3
|   └─ 계란말이 ×1
└─ order 101 / 12:30
    ├── 소주 ×2
    └─ 공기밥 ×1
```

이렇게 하면 최초 주문/추가 주문 시각, POS 주문인지 QR 주문인지, 누가 주문을 받았는지 등을 정확하게 추적할 수 있다.

order_items = 실제 주문된 메뉴

orders가 주문서 머리라면 order_items는 그 안의 실제 메뉴 줄이다. menu_id, 수량, 단가, 할인, 금액, 조리 전송처, 상태 등이 저장된다.

주문 시점 스냅샷

menu_id만 저장하지 않고 item_name과 unit_price도 주문 당시 값으로 저장한다. 나중에 메뉴 이름이나 가격이 바뀌더라도 과거 영수증과 매출은 당시 값 그대로 보여야 하기 때문이다.

예: 2026년에 김치찌개를 10,000원에 판매했고 2027년에 11,000원으로 올려도, 2026년 주문은 계속 10,000원으로 보여야 한다.

5. 메뉴 · 카테고리 · 다국어 · 이미지

menus

판매 가능한 상품의 중심 마스터이다. 김치찌개, 계란말이, 공기밥, 라면사리, 소주, 콜라 등이 각각 하나의 menu가 된다. pos_name은 직원용 이름, price는 기본 판매가, prep_station_id는 준비 장소, is_qr_visible은 QR 노출 여부이다.

menu_categories

찌개, 사이드, 사리, 주류, 음료처럼 POS와 QR 메뉴를 분류한다.

다국어

name_en, name_ja 같은 컬럼을 menus에 계속 추가하지 않는다. menu_translations에 menu_id와 language_code별로 이름과 설명을 저장한다. 카테고리나 옵션도 같은 방식의 번역 테이블을 사용한다.

```
menu_id language name
10 ko 김치찌개
10 en Pork Kimchi Stew
10 ja 일본어 메뉴명
10 zh - CN 중국어 메뉴명
```

menu_images

POS용, QR 썸네일, QR 상세 이미지처럼 용도별 사진을 여러 장 연결할 수 있게 별도 테이블로 관리한다.

6. 옵션과 자동 구성품

modifier = 손님이 선택하는 옵션

맵기 선택, 우동사리 추가, 고기 추가처럼 손님이 직접 선택하는 항목이다. modifier_groups가 옵션 그룹이고 modifier_options가 실제 선택 항목이다. min_select, max_select, is_required를 이용해 필수 선택과 최대 선택 개수를 제어한다.

menu_components = 자동으로 포함되는 구성품

김치찌개 1인분에 공기밥 1개가 무조건 포함된다면 이것은 옵션이 아니다. menu_components에 김치찌개 → 공기밥 1개 관계를 등록한다.

김치찌개 ×3 주문

```
order_item
김치찌개 ×3 / 30,000원 / NORMAL
↓ parent_order_item_id
공기밥 ×3 / 0원 / COMPONENT
```

기본 포함 공기밥은 주방/홀 업무에는 보여주되 손님 영수증에서는 숨길 수 있다. 별도로 추가 주문한 유료 공기밥은 NORMAL 주문이므로 영수증에 정상 표시한다.

7. 주방/홀 조리 전송

prep_stations는 주문을 누가 준비해야 하는지 구분한다. 기본적으로 KITCHEN, HALL, BEVERAGE 같은 장소를 둘 수 있다.

```
주방
- 김치찌개 3
- 계란말이 1

홀
- 기본 공기밥 3
- 소주 2

음료
- 콜라 1
```

order_items에는 주문 시점의 prep_station_id를 스냅샷으로 남긴다. 나중에 메뉴 설정이 바뀌어도 이미 접수된 주문의 조리 경로가 갑자기 변경되지 않게 하기 위해서다.

8. QR 주문 흐름

QR 고객 주문을 즉시 정식 orders로 만들지 않는다. 고객 영역에서 먼저 장바구니와 임시 주문을 만들고 직원 승인 후 정식 주문으로 변환한다.

```
qr_sessions
↓
qr_cart_items
↓
qr_orders (PENDING)
↓ 직원 승인
orders
↓
order_items
```

qr_sessions

고객 브라우저의 QR 이용 세션이다. 언어, 토큰, 마지막 활동시간을 관리하고 필요하면 실제 table_session과 연결한다. 처음에는 session_id가 없어도 메뉴를 둘러보고 장바구니를 만들 수 있게 설계할 수 있다.

승인 처리

직원이 승인할 때 qr_order를 잠그고 PENDING인지 확인한 다음 orders/order_items/options를 생성하고 qr_order를 ACCEPTED로 바꾼다. 이 작업은 하나의 DB 트랜잭션이어야 하며 중복 승인을 방지해야 한다.

핵심: QR 주문은 승인 전에는 고객 임시 주문이고, 승인 후에는 일반 POS 주문과 동일한 orders 체계로 합쳐진다.

9. 계산 · 부분계산 · 복합결제

왜 order에서 payment로 바로 가지 않는가

주문 단위와 실제 계산 단위가 다르기 때문이다. 한 주문의 일부 메뉴만 한 사람이 계산할 수도 있고, 여러 테이블의 주문을 한 번에 계산할 수도 있다.

```
order_items
↓
checkout_items
↓
checkouts
↓
payments
```

checkout_items

이번 계산에 포함되는 order_item과 수량을 지정한다. 김치찌개 4개 중 1개만 먼저 계산하는 것도 가능하다.

payments

실제 받은 돈을 기록한다. checkout 50,000원을 카드 30,000원 + 현금 20,000원처럼 여러 payment로 나눌 수 있어 복합결제가 자연스럽게 지원된다.

결제 취소

원래 결제를 삭제하지 않고 original_payment_id 등으로 원 결제와 취소/환불 거래를 연결한다. 과거 거래 흐름을 보존하기 위해서다.

10. 환불식권 · 할인 · 쿠폰

환불식권과 billing_accounts

회사, 관공서, 병원 등의 환불식권은 즉시 현금이 들어오는 결제와 다르다. payment_method를 환불식권으로 기록하고 billing_account에 어느 기관의 외상인지 연결한다.

할인은 payment가 아니다

```
원래 금액 50,000
할인 - 5,000
결제 대상 45,000
CARD payment 45,000
```

할인액을 음수 payment로 넣지 않는다. checkout_discounts에서 할인 사실을 기록하고 payments에는 실제 받은 돈만 기록한다.

discount_rules

지인 10%, 3,000원 할인, 무료 메뉴 같은 할인 규칙을 정의한다. 최소 주문금액, 최대 할인액, 적용기간, 관리자 승인 필요 여부 등을 관리한다.

쿠폰

```
discount_rules
↓
coupons
↓
coupon_codes
↓
coupon_redemptions
↓
checkout_discounts
```

자체 발행, 네이버, 카카오, 종이쿠폰 등의 채널을 구분할 수 있다. 쿠폰은 발행과 사용 기록을 남기며 사용한 코드를 삭제하지 않는다.

11. 재고 · 레시피 · 재고원장 · 실사

menus와 inventory_items는 다르다

김치찌개는 판매 메뉴이고 돼지고기 · 김치 · 두부 · 육수는 재고 품목이다. 소주처럼 판매 메뉴이면서 동시에 재고 품목인 경우도 있지만 두 개념은 분리한다.

기준 단위

돼지고기는 g, 육수는 ml, 소주는 BOTTLE, 콜라는 CAN, 공기밥은 EA처럼 가장 작은 실용 단위를 기준으로 current_qty를 관리한다.

inventory_units

재고는 CAN으로 관리하지만 매입은 ‘ 5박스 ’로 입력할 수 있다. 예를 들어 콜라 1박스=24캔이면 5박스 입고 시 120 CAN으로 환산한다.

REALTIME과 THEORETICAL

소주 · 콜라처럼 개수 차감이 명확한 품목은 REALTIME으로 즉시 차감한다. 돼지고기 · 김치 · 육수처럼 주문마다 실제 사용량을 재기 어려운 원재료는 THEORETICAL로 레시피를 이용해 예상 사용량을 계산한다.

menu_recipes

```
김치찌개 1인분
- 돼지고기 120g
- 김치 180g
- 두부 50g
- 육수 350ml
```

김치찌개 83인분 판매 시 돼지고기 이론 사용량은 $120 \times 83 = 9,960\text{g}$ 으로 계산할 수 있다.

inventory_transactions = 재고 통장

```
09/01 최초재고 +100 → 100
09/02 매입 +24 → 124
09/02 판매 -3 → 121
09/03 폐기 -1 → 120
```

inventory_items.current_qty는 현재 잔액이고 inventory_transactions는 왜 그 수량이 되었는지를 설명하는 거래 원장이다. 재고 변화 시 둘을 같은 DB 트랜잭션에서 함께 변경한다.

재고 실사

전산 137병, 실제 134병이면 current_qty를 직접 덮어쓰지 않는다. inventory_counts에 전산/실사/차이 -3을 기록하고 확정 시 ADJUSTMENT -3 거래를 남긴다.

12. 매입 · 거래처 · 외상대금

suppliers → purchases → purchase_items

축산업체, 주류업체, 식자재마트, 이마트 등 구매처를 suppliers에 등록한다. 한 번의 입고/구매가 purchases이고 그 안의 개별 품목이 purchase_items이다.

```
○○축산 / 9월 14일 매입
├─ 돼지고기 20kg / 240,000
└─ 소고기 10kg / 300,000
```

입고가 확정되면 purchase_items의 재고수량을 기준으로 inventory_transactions에 PURCHASE 증가를 기록하고 current_qty도 증가시킨다.

외상 매입과 지급

월말 결제 거래처는 매입 당시 UNPAID 상태로 남긴다. 실제 돈을 지급할 때 purchase_payments를 만들고 purchase_payment_allocations로 어느 매입대금을 얼마나 갚았는지 연결한다.

구분: 고객에게 받은 돈은 payments, 거래처에 지급한 돈은 purchase_payments이다. 서로 섞지 않는다.

13. 직원 · 로그인 · 감사로그

staff

OWNER, MANAGER, STAFF 권한을 구분한다. PIN 원문 대신 해시를 저장한다.

staff_sessions

누가 어느 POS 기기에서 로그인 중인지 관리한다. 브라우저가 임의로 보낸 staff_id를 신뢰하지 않고 서버가 인증된 세션/토큰에서 실제 직원 신원을 결정해야 한다.

audit_logs

주문취소, 할인 승인, 재고조정, 결제취소처럼 중요한 행위를 직원 · 기기 · 시간과 함께 기록한다.

19:20 직원A / 주문취소
19:22 매니저 / 지인할인 승인
19:35 사장 / 소주 재고 - 3 조정

14. 실제 손님 한 팀의 전체 흐름

손님 3명이 2번 테이블에 앉고 김치찌개 3, 계란말이 1, 소주 2를 주문한다고 가정한다.

1) dining_tables
2번 테이블

2) table_sessions
session 501 / 3명 / OPEN

3) orders
order 1001 / session 501 / POS

4) order_items
김치찌개 ×3
계란말이 ×1
소주 ×2

5) menu_components 자동 확장
기본 공기밥 ×3 / COMPONENT / 0원

6) 조리 전송
주방: 김치찌개 3, 계란말이 1
홀: 공기밥 3, 소주 2

7) 재고
소주 - 2
공기밥 - 3
원재료는 레시피로 이론 사용량 계산

8) 추가주문
새 order 1002 생성

9) 계산
checkout 생성
할인 적용
카드+현금 등 payments 기록

10) 완료
table_session CLOSED

QR 주문인 경우 3번 앞에 qr_session → qr_cart → qr_order(PENDING) → 직원 승인 단계만 추가되고, 승인 후부터는 일반 POS 주문과 같은 흐름을 사용한다.

15. 웹 POS 개발 시 요청 문장 예시

테이블 화면

현재 dining_tables와 table_sessions 구조를 기준으로 POS 메인 테이블 화면을 만들어줘.
OPEN table_session이 있으면 사용중으로 표시하고 없으면 빈 테이블로 표시해.
같은 party_group의 테이블은 같은 색으로 표시하고,
테이블을 누르면 해당 session의 orders와 order_items를 조회해 주문내역에 보여줘.

테이블 이동

주문 데이터를 수정하지 말고 현재 OPEN table_session의 table_id만 목적 테이블로 변경해줘.
목적 테이블에 다른 OPEN session이 있는지 먼저 검사해줘.

추가 주문

추가 주문은 기존 orders에 품목을 덧붙이지 말고 같은 session_id를 가진 새로운 orders 행을 생성해줘.
주문 시점의 메뉴명, 단가, prep_station을 order_items에 스냅샷으로 저장해줘.

QR 주문 승인

QR 고객 주문은 qr_orders에 PENDING으로 저장해.
직원 승인 시 row lock과 DB transaction을 사용해서 orders/order_items/options로 변환하고,
qr_orders를 ACCEPTED로 변경하면서 accepted_order_id를 저장해.
중복 승인은 불가능하게 해줘.

재고

REALTIME 재고 메뉴가 판매되면 menu_inventory를 조회해 inventory_transactions에 SALE을 기록하고
inventory_items.current_qty를 같은 transaction에서 차감해줘.
THEORETICAL 원재료는 menu_recipes와 실제 판매수량으로 예상 사용량을 계산해줘.

부분/복합 계산

checkout_items를 이용한 부분계산을 만들어줘.
이미 다른 유효 checkout에 배정된 수량을 제외한 미결제 수량만 선택 가능하게 하고,
하나의 checkout에 여러 payments를 넣어 카드+현금 복합결제를 지원해줘.

16. 반드시 지켜야 할 핵심 원칙

원칙	의미
테이블 ≠ 손님	dining_table과 table_session을 분리한다.
추가주문 = 새 order	언제 어떤 주문이 추가됐는지 이력을 보존한다.
옵션 ≠ 구성품	modifier는 고객 선택, component는 자동 포함이다.
QR 승인 전/후 분리	승인 전은 qr_order, 승인 후는 정식 order이다.
주문 ≠ 계산 ≠ 결제	order_item, checkout, payment를 분리한다.
할인 ≠ 결제	할인은 checkout_discounts, 실제 돈은 payments이다.
메뉴 ≠ 재고	판매상품과 원재료/재고품목을 분리한다.
현재수량 + 원장	current_qty와 inventory_transactions를 함께 관리한다.

원칙	의미
삭제보다 상태/반대거래	취소 · 환불 · 조정 이력을 남긴다.
중요 작업은 transaction	주문 승인, 재고 입출고, 계산 등은 원자적으로 처리한다.

최종적으로 이 DB가 목표로 하는 것: 테이블 운영 → POS/QR 주문 → 주방/홀 → 부분 · 복합결제 → 할인/쿠폰 → 재고 → 매입 → 원가/결산까지 하나의 일관된 데이터 흐름으로 연결하는 식당 운영 시스템.